

StockSyncServices

Architecture, Complete Workflow & Operating Pipeline

A microservice-based inventory platform of three independently-deployable NestJS services — **user-service**, **warehouse-service** and **supplier-service** — each backed by its own MongoDB Atlas database and coordinated over RabbitMQ.

This document explains how the system fits together, walks through one complete request from registration to a synced stock level, and lists every endpoint, event, and config value needed to run and extend it.

Services	user-service (8000) · warehouse-service (8001) · supplier-service (8002)
Datastore	MongoDB Atlas — one database per service (database-per-service pattern)
Messaging	RabbitMQ — user_events_queue, warehouse_events_queue
Auth	JWT (shared secret), role-based guards: WAREHOUSE_MANAGER, SUPPLIER, CUSTOMER
Status	Builds clean · 60 unit tests passing · full flow verified live end-to-end

1. What this system does

StockSyncServices links three independent business roles — a **warehouse manager**, a **supplier**, and (in future) a **customer** — through a shared identity service and an event-driven stock-sync pipeline. A supplier's product quantities are kept mirrored into every warehouse that has approved that supplier's capacity request, without the services ever calling each other directly over HTTP for that data.

Design principles

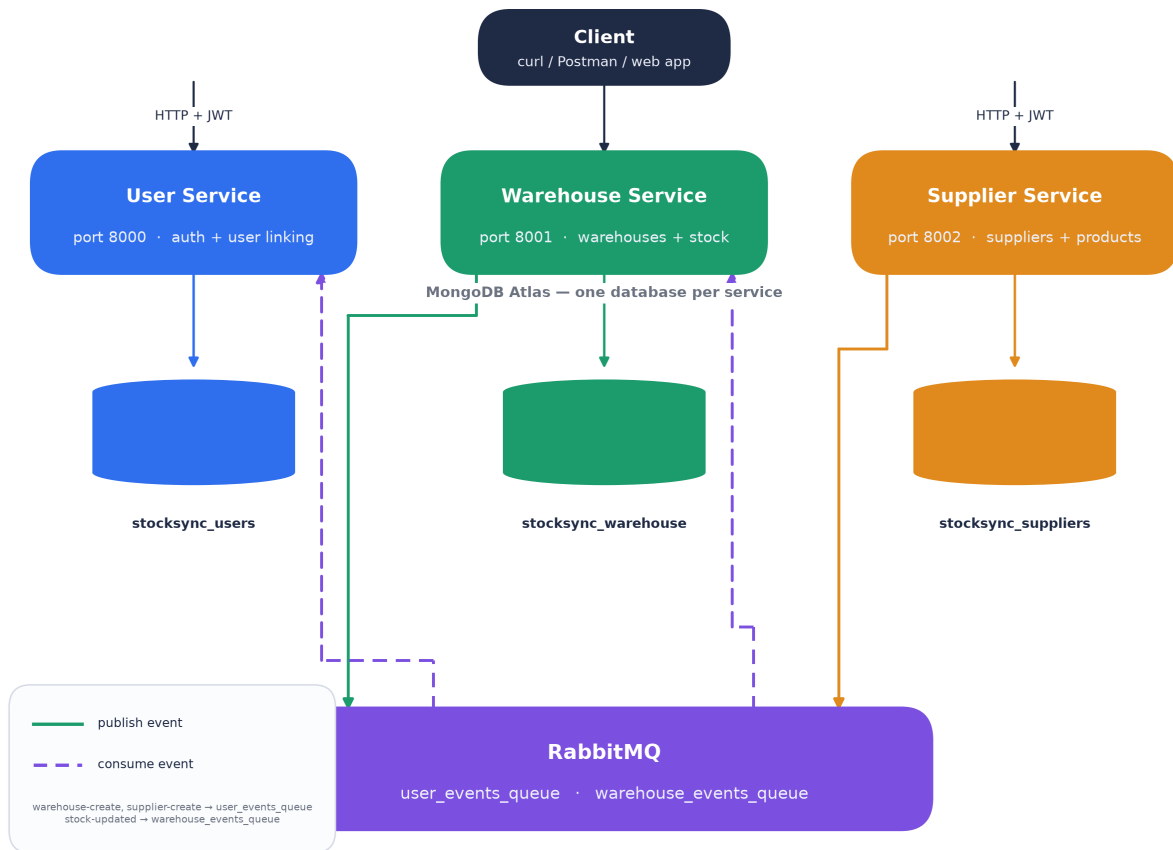
- **Database per service** — user-service, warehouse-service and supplier-service each own a private MongoDB Atlas database (stocksync_users / stocksync_warehouse / stocksync_suppliers). No service reads another's collections directly.
 - **Events, not synchronous calls, for cross-service writes** — a warehouse or supplier never calls another service's HTTP API to update its data; it publishes a fact to RabbitMQ and lets the owner apply it.
 - **One identity, shared by reference** — a user's Mongo _id issued by user-service is the same id used everywhere as admin, supplierId, and userId across the other two services and inside JWTs.
 - **Independent deployability over shared code** — the JWT/roles guards are intentionally duplicated per service rather than pulled into a shared library, so each service can still be built, tested and deployed on its own.
-

2. System architecture

Every client request enters through one of the three services directly (there is no API gateway yet — see Section 8). Each service owns its own database. The only communication between services is asynchronous, over two RabbitMQ queues.

StockSyncServices — System Architecture

Three independently-deployable NestJS microservices, one MongoDB Atlas database each, linked by RabbitMQ



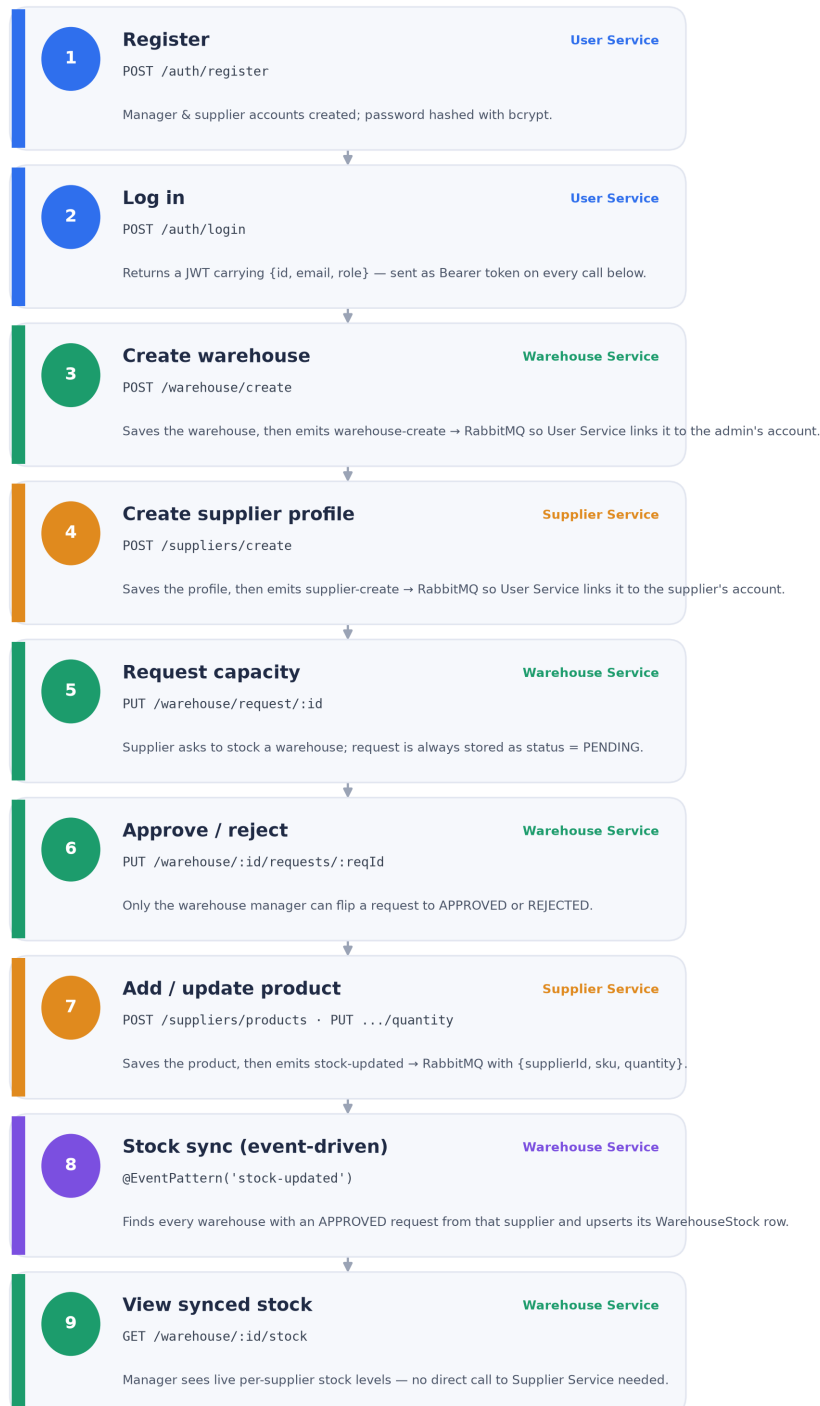
Why RabbitMQ instead of a direct HTTP call? When warehouse-service creates a warehouse, user-service needs to know so it can add the warehouse to the admin's account — but warehouse-service shouldn't need to know user-service's address, retry logic, or be blocked waiting on it. Publishing warehouse-create and moving on keeps the two services decoupled: if user-service is briefly down, the message waits durably in the queue instead of the warehouse-creation request failing.

3. Complete end-to-end workflow

This is the exact sequence exercised in live testing against the real Atlas cluster and RabbitMQ — registering both roles, creating a warehouse, onboarding a supplier, approving a capacity request, and watching a product's stock sync into the warehouse automatically.

StockSyncServices — End-to-End Workflow

One full pass through the pipeline, in the order a client actually calls it



3.1 The stock-sync event, in detail

Step 8 is the core mechanism the whole system exists for. When a supplier's product quantity changes, here is exactly what happens inside warehouse-service's event handler (`WarehouseService.handleStockUpdated`):

- Reads { `supplierId`, `sku`, `productName`, `quantity` } off the `warehouse_events_queue`.
- Queries every Warehouse document whose `requests` array has an entry for that `supplierId` with status: 'APPROVED'.
- For each matching warehouse, upserts a `WarehouseStock` row keyed on (`warehouseId`, `supplierId`, `sku`) with the new quantity.
- A supplier with no approved warehouse yet simply produces zero matches — the event is a no-op until a manager approves them.

This means stock only ever flows to warehouses that explicitly opted in by approving the supplier, and a supplier can update their catalog freely before any warehouse relationship exists — the sync is naturally idempotent and safe to replay.

4. API reference

4.1 user-service — port 8000

user-service exposes no HTTP endpoints for warehouse/supplier linking — that happens purely by consuming warehouse-create and supplier-create events (Section 5).

4.2 warehouse-service — port 8001

Method	Endpoint	Role	Purpose
GET	<code>/warehouse/get</code>	WAREHOUSE_MANAGER	List warehouses owned by the caller.
POST	<code>/warehouse/create</code>	WAREHOUSE_MANAGER	Create a warehouse.
PUT	<code>/warehouse/request:warehouseId</code>	SUPPLIER	Request capacity (always saved as PENDING).
PUT	<code>/warehouse/:warehouseId/requests/requestId</code>	WAREHOUSE_MANAGER	Approve or reject a request.
GET	<code>/warehouse/:warehouseId/stock</code>	WAREHOUSE_MANAGER	View synced stock for that warehouse.

4.3 supplier-service — port 8002

Method	Endpoint	Role	Purpose
POST	<code>/suppliers/create</code>	SUPPLIER	Create the caller's supplier profile.
GET	<code>/suppliers/get</code>	SUPPLIER	Fetch the caller's supplier profile.

Method	Endpoint	Role	Purpose
POST	/suppliers/products	SUPPLIER	Add a product (emits stock-updated).
GET	/suppliers/products	SUPPLIER	List the caller's products.
PUT	/suppliers/products/:productId/quantity	SUPPLIER	Update quantity (emits stock-updated).

5. Event catalogue

Event	Producer	Queue	Consumer	Payload
warehouse-create	warehouse-service	user_events_queue	user-service	{ warehouseId, userId }
supplier-create	supplier-service	user_events_queue	user-service	{ supplierProfileId, userId }
stock-updated	supplier-service	warehouse_events_queue	warehouse-service	{ supplierId, sku, productName, quantity }

All three events are fire-and-forget (`ClientProxy.emit`, not `send`) — the publisher never waits for or depends on the consumer's response. Handlers log and swallow their own errors so one bad event can never crash the listening service.

6. Data model — one collection set per database

6.1 stocksync_users

Collection	Key fields	Notes
users	name, email, password (bcrypt), role, warehouses[], supplierProfile	role $\in$ {WAREHOUSE_MANAGER, SUPPLIER, CUSTOMER}. warehouses[] and supplierProfile are populated only by consumed events.

6.2 stocksync_warehouse

Collection	Key fields	Notes
warehouses	admin, name, location, capacity, requests[]	requests[] holds {supplierId, requestCapacity, status}; status defaults to PENDING and can only change via the approve/reject endpoint.
warehousestocks	warehouseId, supplierId, sku, productName, quantity	Read-model populated only by the stock-updated event handler; unique on (warehouseId, supplierId, sku).

6.3 stocksync_suppliers

Collection	Key fields	Notes
suppliers	userId, businessName, contactEmail, contactPhone, address	1:1 with a SUPPLIER-role user; userId is that user's own _id, not a new identity.
products	supplierId, sku, name, unitPrice, quantity	unique on (supplierId, sku); every create/update emits stock-updated.

7. Configuration

Every service loads its own .env via @nestjs/config — nothing is hardcoded in source. .env is git-ignored; .env.example is committed as a template.

Variable	Meaning
PORT	HTTP port for that service (8000 / 8001 / 8002).
MONGO_URI	That service's own Atlas connection string — different database name per service.
JWT_SECRET	Shared across all three services so a token minted by user-service verifies everywhere.
JWT_EXPIRES_IN	Token lifetime (default 1h).
RABBITMQ_URL	e.g. amqp://localhost:5672 — add ?frameMax=8192 if your broker is RabbitMQ 4.x.
USER_EVENTS_QUEUE	Queue user-service listens on; warehouse/supplier-service publish to it.
WAREHOUSE_EVENTS_QUEUE	Queue warehouse-service listens on; supplier-service publishes to it.

8. Testing & verification

Unit tests — 60 passing across all three services

- **user-service (22 tests)** — AuthService (register/login, duplicate email, wrong password), UsersService (event handlers), both guards.
- **warehouse-service (20 tests)** — WarehouseService (create, request, approve/reject, stock sync, malformed-id rejection), controller wiring, both guards.
- **supplier-service (18 tests)** — SuppliersService (profile + product CRUD, duplicate SKU/profile), controller wiring, both guards.

All Mongo models and RabbitMQ ClientProxy instances are mocked in these tests — no network or database access happens during npm test.

Live end-to-end verification

Beyond unit tests, the full pipeline above was run against the real Atlas cluster and a local RabbitMQ instance: register → login (both roles) → create warehouse → create supplier profile → request capacity → approve → add product → confirm synced stock → update quantity → confirm re-sync. Negative paths were exercised too: duplicate registration (401), wrong password (401), missing token (401), and wrong role on a guarded route (403).

9. Running it

Option A — Docker Compose (recommended once Docker is installed)

```
docker compose up --build
```

Starts RabbitMQ plus all three services, wired to their own .env files. MongoDB stays on Atlas — it is not containerized.

Option B — run each service directly

Requires a running RabbitMQ (locally via Homebrew — `brew services start rabbitmq` — or any reachable broker).

- `cd user-service && npm run start:dev (port 8000)`
 - `cd warehouse-service && npm run start:dev (port 8001)`
 - `cd supplier-service && npm run start:dev (port 8002)`
-

10. Future roadmap

Deliberately out of scope for this pass — listed here so the next iteration has a clear starting point:

- Refresh tokens + token blacklist/revocation.
- Rate limiting on auth endpoints.
- A notification-service reacting to stock-sync and approval events.
- Structured logging and health-check endpoints per service.
- CI (GitHub Actions) running lint + test on every push.
- An API gateway or documented single entry point for clients.
- Revisit shared-library vs. per-service duplication for the JWT/roles guards as the team grows.